HearDesk

SonarQube Code Quality Report

Code checks • Test coverage • Findings and fixes

Website feedback. Ready for your team.

A personal full-stack engineering project

Prepared by Muhammad Danu Susyanto

Version 1.0 • 13 October 2026

Portfolio edition describing the implemented app, with test examples and dated results.

Contents

Contents 2
1. About this report 3
2. What the terms mean. 3
3. Recorded quality result 3
4. How analysis and test results are collected. 4
5. Changes across recorded checks 4
6. Findings and their fixes 5
7. Why the dialog attribute was kept 5
8. How test coverage improved 6
9. Reading the metrics correctly. 6
10. Repeating the checks and updating the report 6
11. Evidence files and limits 6

1. About this report

This document explains the HearDesk features and code reviewed for this portfolio. The results describe the recorded version and test environment. They do not represent a customer contract, independent certification or measured business results.

SonarQube checks source code for potential problems and imports test coverage results. This report records project heardesk on branch master. The results retrieved for the analysis dated 13 October 2026 at 03:37:04 UTC (10:37:04 WIB) showed Quality Gate OK.

The available SonarQube API response did not identify the exact Git commit analyzed. The documentation uses local commit afb4f54a83eafcd0f10232adbbeeae92292c293a. These identifiers are recorded separately rather than assumed to refer to the same source revision.

2. What the terms mean

Table 1. What the terms mean

Term	Meaning
Static analysis	Checking source code for rule violations without proving every possible runtime behavior.
Quality Gate	A set of pass/fail rules configured in SonarQube, such as minimum coverage and no new issues.
Coverage	How much executable code and its decision paths the tests exercised. It is not a percentage of guaranteed correctness.
New code	Code that SonarQube considers new or changed in the configured comparison period. It is not always just the latest commit.
Duplication	Code blocks the analyzer identifies as repeated.
Issue	A finding raised by an analysis rule. It needs review to establish whether it represents a real problem.
False Positive	A finding reviewed as an incorrect warning about valid code. The reason should be recorded.
Security hotspot	Code marked for security review. It is not automatically a confirmed vulnerability.
LCOV	A file format used to tell SonarQube which lines and branches were executed by tests.

3. Recorded quality result

The retrieved result was Passed (API status OK). New-code coverage was 80.4%, overall coverage was 80.6%, new issues were 0, and new duplicated lines were 0.0%. Reported bugs, vulnerabilities and security hotspots were also 0.

The figures describe that retrieved snapshot. They do not certify security or show that every runtime path works. The 80.4% new-code result is only slightly above the configured minimum of 80%.

Table 2. Recorded quality result

Check	Required value	Recorded value	Outcome
New-code coverage	At least 80%	80.4%	Passed
New duplicated lines	At most 3%	0.0%	Passed
New issues	0	0	Passed

Check	Required value	Recorded value	Outcome
New security hotspots reviewed	100% when applicable	No numeric value was returned. The condition status was OK.	Passed

4. How analysis and test results are collected

sonar-project.properties tells SonarQube to analyze src, scripts and public, and to treat tests as test code. Generated Prisma code is excluded. Coverage is imported from coverage/lcov.info and coverage/integration/lcov.info.

The two coverage files describe unit/component and database integration runs. Their percentages must not simply be added. SonarQube combines the covered code when processing the reports.

Jenkins runs automated build and analysis work. In the approved configuration, the scanner does not wait for the final Quality Gate result. A successful Jenkins job and a Passed SonarQube gate are therefore separate facts that both need checking. Test or scanner errors still fail the job.

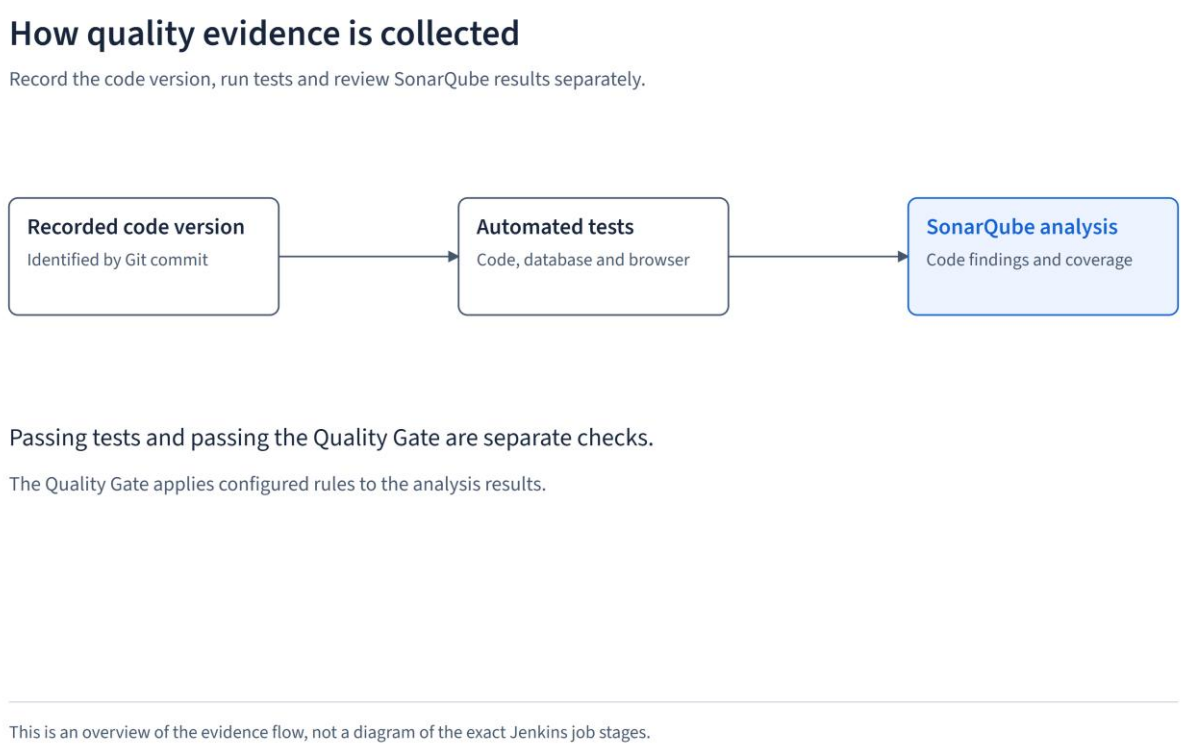


Figure 1. How analysis and test results are collected

5. Changes across recorded checks

These rows show the gate and metrics retrieved during the review sessions. A current Overview status can differ from an Activity entry recorded earlier, for example after issue review. This table is a record of retrieved checks, not an independently verified history of every status change.

Table 3. Changes across recorded checks

Analysis time (UTC)	New coverage	New issues	Retrieved gate	What remained
12 Oct 2026 11:43:09	80.0%	9	ERROR	Coverage met its threshold, but nine code findings failed the gate.
13 Oct 2026 03:16:22	79.9%	1	ERROR	Eight fixes were recognized. Coverage was below 80%, and the dialog-attribute finding remained.
13 Oct 2026 03:37:04	80.4%	0	OK	All reported gate conditions passed in the retrieved snapshot.

6. Findings and their fixes

The rule IDs below identify SonarQube checks. Their descriptions explain why each change was made. Syntax examples are included so a developer can locate the exact change.

Table 4. Findings and their fixes

Rule / count	Meaning of the finding	Change made
S3776 / 1	The submit function's cognitive complexity score was 19, above the allowed 15. This score estimates how difficult the control flow is to follow.	Moved multipart reading and field checks into feedback-form-request.ts while keeping the same validation order, limits and errors.
S6582 / 2	Longer null checks could use optional chaining, the ?. syntax that safely handles missing values.	Used item?.id and report?.trackingHash while keeping the same report-identity and missing-target checks.
S7718 / 2	The catch error variable did not follow the configured naming convention.	Renamed problem to error_ without changing error handling.
S6772 / 3	Spacing between label text and a form control was unclear in JSX markup.	Wrapped the label text in a span without changing the control's accessible name.
S6747 / 1	The analyzer did not recognize the dialog's closedby attribute.	Kept the valid browser behavior and supplied a False Positive review rationale.

7. Why the dialog attribute was kept

The HTML attribute closedby="any" allows a dialog to close when the user clicks outside it, presses Escape or uses its close button. MDN documents the attribute, and HearDesk's browser tests check Escape and click-outside behavior.

Removing this attribute just to clear the warning would remove an intended interaction. The recommendation was to review the finding as a False Positive rather than change the dialog behavior.

The retrieved result showed no open issues. The available response did not include a complete issue-resolution history, so this document does not claim who changed the finding or which action produced the final status.

Reference: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/dialog#closedby.

8. How test coverage improved

Five additional tracking-page component tests check an incomplete code, a correctly formatted request, reply and status refresh, recovery from an error, and old responses arriving after a newer lookup. They verify useful behavior rather than just checking that source text exists.

Tracking-component coverage rose from 33 of 46 lines and 15 of 22 branches to all 46 lines and 22 branches. A branch is a different route through a decision, such as the valid-code and invalid-code paths.

The locally calculated combined LCOV estimate rose from 79.73% to 80.12%. SonarQube later reported 80.4% on new code and 80.6% overall. Those numbers use different scopes and calculations and should not be presented as interchangeable.

The recorded follow-up passed 269 unit/component tests and 90 integration tests. No application behavior was changed to raise coverage during that follow-up.

9. Reading the metrics correctly

Table 5. Reading the metrics correctly

Metric	Value	What it tells the reader
Overall coverage	80.6%	Coverage across the analyzed code, not only the code in the comparison period.
New-code coverage	80.4%	Coverage of code selected by SonarQube's new-code definition.
New duplicated lines	0.0%	No measured duplication in that new-code scope. It does not establish perfect architecture.
Open issues	0	The query for OPEN and CONFIRMED findings returned none.
Bugs / vulnerabilities	0 / 0	No such findings were reported by this analysis. Runtime testing and security review remain separate.
Security hotspots	0	No hotspots were reported in the retrieved snapshot.

10. Repeating the checks and updating the report

Run npm run test:coverage:all and retain both coverage files. Check every test result, run the Sonar scanner with the project's configuration, then retrieve the gate, metrics and issues for the intended branch.

Record the analysis time and exact commit when available. Explain any False Positive decision using source evidence and behavior tests. Do not weaken thresholds or remove source from analysis merely to obtain a better result.

As the code grows, add tests for meaningful missing behavior. Keep a margin above the minimum coverage rather than depending on one rounded value.

11. Evidence files and limits

The saved remote result is documentation/source/evidence/sonarqube.json. Configuration and test references are sonar-project.properties, package.json, vitest.config.ts, vitest.integration.config.ts, tests/components/feedback-tracking.test.tsx and tests/feedback-form-request.test.ts.

This report excludes access tokens, infrastructure credentials and private user details. It describes static code checks and imported test coverage. The separate End-to-End UI Test Report explains browser behavior checks. Neither report is a claim that all security risks have been eliminated.